

CSE 4403: Algorithms

Lecture17

Divide and Conquer - 2

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Maximum Sub-array Sum

Problem Statement:

Given an array of integers (can be negative), find the contiguous sub-array with the largest sum.

Largest Subarray Sum Problem

-2	-3	4	-1	-2	1	5	-3
0	1	2	3	4	5	6	7

$$4 + (-1) + (-2) + 1 + 5 = 7$$

Maximum Contiguous Array Sum is 7

Divide and Conquer Idea

Split the array at the midpoint. **The maximum sub-array must be in one of three places:**

- Entirely in the left half
- Entirely in the right half
- Crossing the midpoint (starts in left, ends in right)

The first two are solved recursively. The crossing case is handled directly.

Maximum Sub-array Sum

```
solve(arr, l, r):  
    if l == r:  
        return arr[l]          // base case  
  
    mid = (l + r) / 2  
  
    left_max    = solve(arr, l, mid)  
    right_max   = solve(arr, mid+1, r)  
    cross_max   = findCrossing(arr, l, mid, r)  
  
    return max(left_max, right_max, cross_max)
```

Split

Divide into independent subproblems

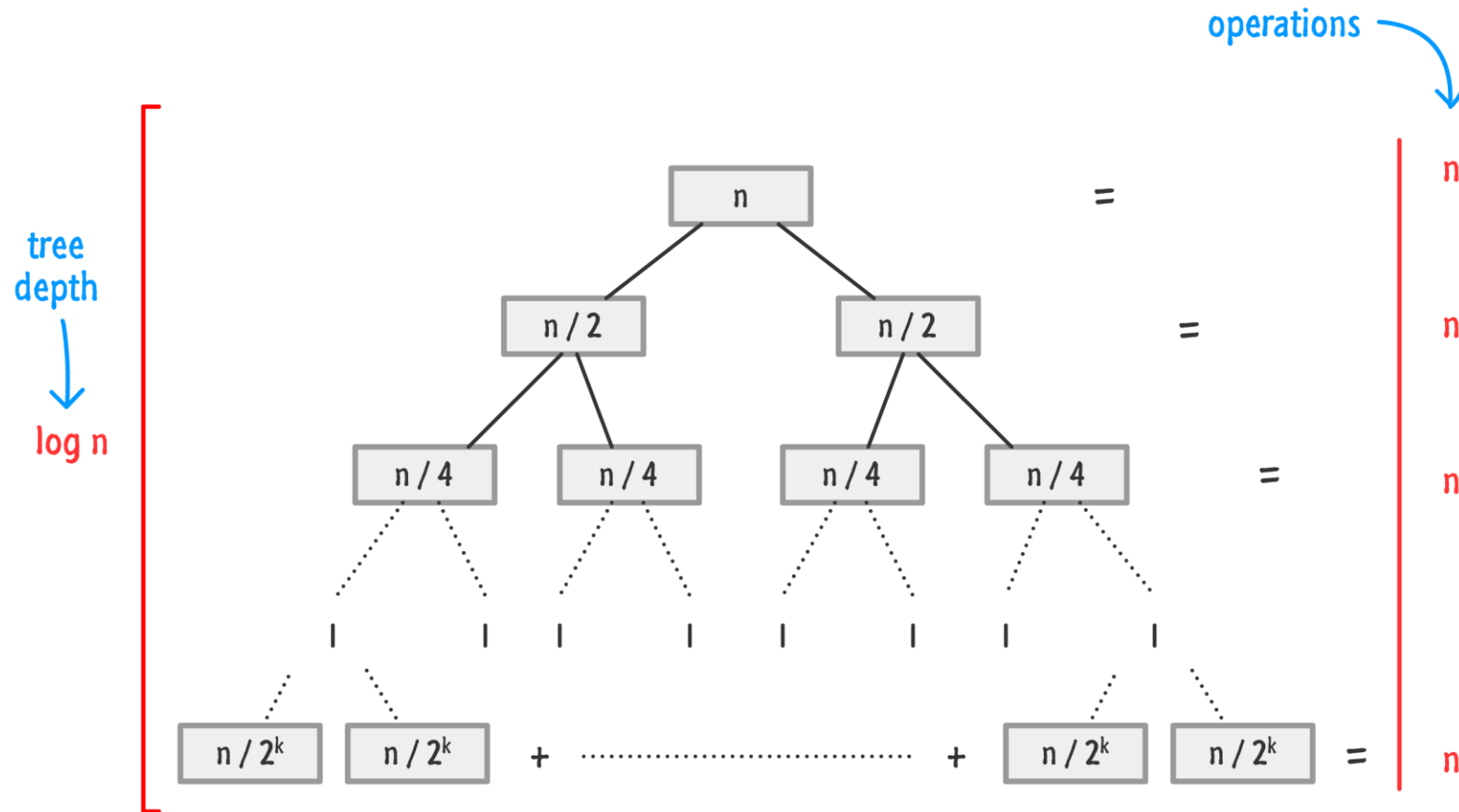
Solve

Recurse on each part

Combine

Merge results into final answer

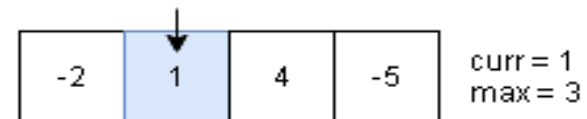
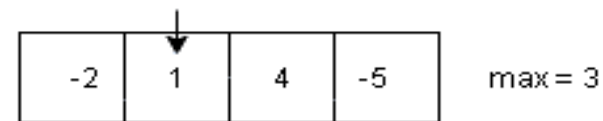
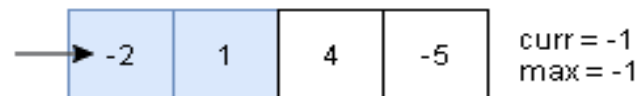
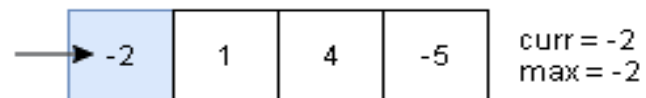
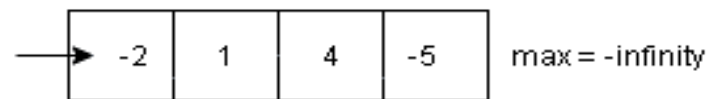
Tree



Note: Starting Index Matters

Array

-2	1	4	-5
----	---	---	----



Cross Sum



```
int findCrossing(vector<int>& arr, int l, int mid, int r)
{
    int left_sum = INT_MIN, sum = 0;
    for (int i = mid; i >= l; i--) {
        sum += arr[i];
        left_sum = max(left_sum, sum);
    }

    int right_sum = INT_MIN; sum = 0;
    for (int i = mid+1; i <= r; i++) {
        sum += arr[i];
        right_sum = max(right_sum, sum);
    }

    return left_sum + right_sum;
}
```

Time Complexity: Remember

Given a problem of size n

Divide into a subproblems

Each with size $\frac{n}{b}$

Where $a \geq 1$ and $b > 1$

Solve each subproblem recursively

Combine the solutions

Running time: $T(n) = aT\left(\frac{n}{b}\right) + [merge]$

Longest Nice Substring

Problem Statement:

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase

- aAbBAAA – Nice string
- Abc – Not nice
- abBxCDScsedEad - ??

Longest Nice Substring

Problem Statement:

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase

- aAbBAAA – Nice string
- Abc – Not nice
- a**b**x**CD****Scsed**Ead – Not nice (But there are substrings, which are nice)

Longest Nice Substring

Problem Statement:

A string s is nice if, for every letter of the alphabet that s contains, it appears both in uppercase and lowercase

- aAbBAAA – Nice string
- Abc – Not nice
- a**b**x**CDS**cs**dE**ad – Not nice (But there are substrings, which are nice)

Our task is to pick the longest such substring!

Trivial Solution

- Find all possible substring
- Check which one of them are nice
- Print the one with highest length
- Number of all possible nonempty substring: $n(n+1)/2$
- $O(n^2)$

a**b**Bx**CD**Scsd**E**ad

Substrings:

a, ab, abB...	total n
b, bB, bBx...	total n-1
...	

$n+(n-1)+....2+1$

$=n(n+1)/2$

Trivial Solution

- If the string length is less than 2, return empty string.
- Keep the track of occurrence of every alphabets.

(To check whether a letter have its counter-letter in the string or not)

Trivial Solution

- If the string length is less than 2, return empty string.
- Keep the track of occurrence of every alphabets (To check whether a letter have its counter-letter in the string or not)

```
if(s.length()<2) return "";

set <char> ch_set;
int len = s.length();

for(int i = 0; i<len; i++){
    ch_set.insert(s[i]); // In case of set
    //ar[s[i] - 'a'] = 1; //In case of array
}
```

Solution

```
string longestNiceSubstring(string s) {
    return solve(s, 0, s.size() - 1);
}

string solve(string& s, int l, int r) {
    if (r - l < 1) return "";          // need at least 2 chars
    set<char> present(s.begin() + l, s.begin() + r + 1);
    for (int i = l; i <= r; i++) {
        char c = s[i];
        // check if its pair is missing
        if (present.find(tolower(c)) == present.end() ||
            present.find(toupper(c)) == present.end()) {

            string left  = solve(s, l, i - 1);
            string right = solve(s, i + 1, r);
            return left.size() >= right.size() ? left : right;
        }
    }
    return s.substr(l, r - l + 1);    // whole segment is nice
}
```

How does the tree look like?